

Chapter 1. Introducing C#

The C# programming language (pronounced “see sharp”) is used for many kinds of applications, including websites, cloud-based systems, artificial intelligence, IoT devices, desktop applications, embedded controllers, mobile apps, games, and command-line utilities. C#, along with the supporting runtime, libraries, and tools known collectively as .NET, has been center stage for Windows developers for over 20 years. Today, .NET is cross-platform and open source, enabling applications and services written in C# to run on operating systems including Android, iOS, macOS, and Linux, as well as on Windows.

Every new version of C# has enhanced developer productivity. For example, the most recent versions include new pattern-matching features to make our code more expressive and succinct. Primary constructors and collection expressions help to reduce verbosity in some common scenarios. Various type-system enhancements allow our code to express its requirements and characteristics in more detail, enabling us to write more flexible libraries, and to enjoy better compile-time diagnostics.

C# 11.0 and 12.0 have gained performance-oriented features including generic math, and improved control over memory handling for performance-sensitive low-level code. Every new .NET release has improved execution speed, but there have also been significant reductions in startup times, memory footprint, and binary size. This, along with improved support for containerization, enhances .NET’s fit for modern cloud development. There have also been significant improvements for cross-platform client-side development, thanks to Blazor and .NET MAUI (Multi-platform App UI). .NET has supported ARM and WebAssembly (WASM) for many years, but continuous recent improvements for those targets are important for cloud, mobile, and web development.

C# and .NET are open source projects, although it didn't start out that way. In C#'s early history, Microsoft guarded all of its source code closely; however, in 2014, the [.NET Foundation](#) was created to foster the development of open source projects in the .NET world. Many of Microsoft's most important C# and .NET projects are now under the foundation's governance (in addition to many non-Microsoft projects). This includes [the C# language standard](#), [Microsoft's C# compiler](#), and also the [.NET runtime and libraries](#). Today, pretty much everything surrounding C# is developed in the open, with code contributions from outside of Microsoft being welcome. New language feature proposals are managed on GitHub, enabling community involvement from the earliest stages.

Why C#?

Although there are many ways you can use C#, other languages are always an option. Why might you choose C# over those? It will depend on what you need to do and what you like and dislike in a programming language. I find that C# provides considerable power, flexibility, and performance and works at a high enough level of abstraction that I don't expend vast amounts of effort on little details not directly related to the problems my programs are trying to solve.

Much of C#'s power comes from the range of programming techniques it supports. For example, it offers object-oriented features, generics, and functional programming. It supports both dynamic and static typing. It provides powerful list- and set-oriented features, thanks to Language Integrated Query (LINQ). It has intrinsic support for asynchronous programming. Moreover, the various development environments that support C# all offer a wide range of productivity-enhancing features.

C# provides options for balancing ease of development against performance. The runtime has always provided a garbage collector (GC) that frees developers from much of the work associated with recovering memory that the program is no longer using. A GC is a common feature in modern programming languages, and while it is a boon for most programs, there are some specialized scenarios where its performance implications are problematic. That's why C# also enables more explicit mem-

ory management, giving you the option to trade ease of development for runtime performance but without the loss of type safety. This makes C# suitable for certain performance-critical applications that for years were the preserve of less safe languages such as C and C++.

Languages do not exist in a vacuum—high-quality libraries with a broad range of features are essential. Some elegant and academically beautiful languages are glorious right up until you want to do something prosaic, such as talking to a database or determining where to store user settings. No matter how powerful a set of programming idioms a language offers, it also needs to provide full and convenient access to the underlying platform's services. C# is on very strong ground here, thanks to its runtime, built-in class libraries, and extensive third-party library support.

.NET encompasses both the runtime and the main class libraries that C# programs use. The runtime part is called the *Common Language Runtime* (usually abbreviated to CLR) because it supports not just C# but any .NET language. Microsoft also offers Visual Basic, F#, and .NET extensions for C++, for example. The CLR has a *Common Type System* (CTS) that enables code from multiple languages to interoperate freely, which means that .NET libraries can normally be used from any .NET language—F# can consume libraries written in C#, C# can use Visual Basic libraries, and so on.

There is an extensive set of class libraries built into .NET. These have gone by a few names over the years, including Base Class Library (BCL), Framework Class Library, and framework libraries, but Microsoft now seems to have settled on *runtime libraries* as the name for this part of .NET. These libraries provide wrappers for many features of the underlying operating system (OS), but they also provide a considerable amount of functionality of their own, such as collection classes and JSON processing.

The .NET runtime class libraries are not the whole story—many other systems provide their own .NET libraries. For example, there are libraries that enable C# programs to use popular cloud services. As you'd expect, Microsoft provides comprehensive .NET libraries for working with services in its Azure cloud platform. Likewise, Amazon provides a fully featured development kit for using Amazon Web Services (AWS) from C#

and other .NET languages. And libraries do not have to be associated with particular services. There's a large ecosystem of .NET libraries, some commercial and some free, including mathematical utilities, parsing libraries, and user interface (UI) components, to name just a few. Even if you get unlucky and need to use an OS feature that doesn't have any .NET library wrappers, C# offers various mechanisms for working with other kinds of APIs, such as the C-style APIs available in Win32, macOS, and Linux, or APIs based on the Component Object Model (COM) in Windows.

In addition to libraries, there are also numerous application frameworks. .NET has built-in frameworks for creating web apps and web APIs, desktop applications, and mobile applications. There are also open source frameworks for various styles of distributed systems development, such as high-volume event processing with [Reactor](#) or high-availability globally distributed systems with [Orleans](#).

Finally, with .NET having been around for over two decades, many organizations have invested extensively in technology built on this platform. So C# is often the natural choice for reaping the rewards of these investments.

In summary, C# gives us a strong set of abstractions built into the language, a powerful runtime, and easy access to an enormous amount of library and platform functionality.

Managed Code and the CLR

C# was the first language designed to be a native in the world of the CLR. This gives C# a distinctive feel. It also means that if you want to understand C#, you need to understand the CLR and the way in which it runs code.

For years, the most common way for a compiler to work was to process source code and to produce output in a form that could be executed directly by the computer's CPU. Compilers would produce *machine code*—a series of instructions in whatever binary format was required by the kind of CPU the computer had. This is sometimes referred to as *native code*, because it's the language the CPU inherently understands. Many compilers

still work this way, but although we can compile C# into machine code, we often don't. This is optional because C# uses a model called *managed code*.

With managed code, the compiler does not generate the machine code that the CPU executes. Instead, the compiler produces a form of binary code called the *intermediate language* (IL). The executable binary is produced later, usually, although not always, at runtime. The use of IL enables features that are hard or even impossible to provide under the more traditional model.

Perhaps the most visible benefit of the managed model is that the compiler's output is not tied to a single CPU architecture. For example, the Intel and AMD CPUs used in many modern computers support both 32-bit and 64-bit instruction sets (known, respectively, for historical reasons as x86 and x64). With the old model of compiling source code into machine language, you'd need to choose which of these to support, building multiple versions of your component if you need to target more than one. But with .NET, you can build a single component that can run without modification in either 32-bit or 64-bit processes. The same component could even run on completely different architectures such as ARM (a processor architecture widely used in mobile phones, Macs with *Apple Silicon*, and also in tiny devices such as the Raspberry Pi). With a language that compiles directly to machine code, you'd need to build different binaries for each of these, or in some cases you might build a single file that contains multiple copies of the code, one for each supported architecture. With .NET, you can compile a single component that contains just one version of the code, and it can run on any of them. It would even be able to run on platforms that weren't supported at the time you compiled the code if a suitable runtime became available in the future. (For example, .NET components written years before Apple released its first ARM-based Macs can run without relying on the [*Rosetta*](#) translation technology that normally enables older code to work on the newer processors.)

More generally, any kind of improvement to the CLR's code generation—whether that's support for new CPU architectures or just performance improvements for existing ones—instantly benefits all .NET languages. For

example, older versions of the CLR did not take advantage of the vector processing extensions available on modern processors, but the current versions will now often exploit these when generating code for loops. All code running on current versions of .NET benefits from this, including components that were compiled years before this enhancement was added.

The exact moment at which the CLR generates executable machine code can vary. By default, it uses an approach called *just-in-time* (JIT) compilation, in which each individual function's machine code is generated the first time it runs. However, it doesn't have to work this way. One of the runtime implementations, called Mono, is able to interpret IL directly without ever converting it to runnable machine language, which is useful on platforms such as iOS where legal constraints may prevent JIT compilation. The .NET Software Development Kit (SDK) also provides ways to build precompiled code alongside the IL. This *ahead-of-time* (AOT) compilation can improve an application's startup time.

NOTE

Generation of executable code can still happen at runtime. The CLR's *tiered compilation* feature may choose to recompile a method dynamically to optimize it better for the ways it is being used at runtime, and it can do this even when you use AOT compilation because the IL is still available at runtime.

The .NET SDK offers a more extreme option called *Native AOT*. Instead of combining IL and native code, applications built with Native AOT contain only native code.¹ Runtime features including the garbage collector and any runtime library components the application requires are included in the output, making Native AOT applications completely self-contained. (The bundled runtime features do not include the JIT compiler. It's unnecessary, because all IL is compiled to machine language at build time.) Unlike with other .NET compilation models, Native AOT applications do not need a copy of the .NET runtime to be either preinstalled or shipped alongside the application code. Not all applications can use Native AOT, because some .NET libraries exploit the CLR's ability to JIT compile code by generating new code at runtime, so these don't work (or have limited

functionality) on Native AOT. But in cases where it is applicable, it can dramatically lower startup times for applications that are able to use it.

Managed code has ubiquitous type information. The .NET runtime requires this to be present to enable certain features. For example, .NET offers various automatic serialization services, in which objects can be converted into binary or textual representations of their state, and those representations can later be turned back into objects, perhaps on a different machine. This sort of service relies on a complete and accurate description of an object's structure, something that's guaranteed to be available in managed code. Type information can be used in other ways. For example, unit test frameworks can use it to inspect code in a test project and discover all of the unit tests you have written. These kinds of features typically rely on the CLR's *reflection* services, which are the topic of [Chapter 13](#). However, Native AOT imposes some restrictions—full type information is available at the point where Native AOT starts to generate native code, but unless it can deduce that your code will rely on that type information at runtime, it will often trim some of this out. This makes the compiled output significantly smaller, which can improve startup times, but it also means that by default, the final output might have an incomplete picture, which is another reason not all libraries work with Native AOT. However, the .NET team intends to make Native AOT viable for as many applications as possible, which is why the last few versions of the .NET SDK have added compile-time code generation features that can reduce the reliance on runtime reflection. For example, it can generate code enabling the JSON libraries described in [Chapter 15](#) to perform serialization without using reflection. This still relies on full type information being available for all .NET code during the build process; it just enables it to be dropped from the final build output.

Although C#'s close connection with the runtime is one of its main defining features, it's not the only one. There's a certain philosophy underpinning C#'s design.

C# Prefers Generality to Specialization

C# favors general-purpose language features over specialized ones. C# is now on its 12th major version, and with every release, the language's designers had specific scenarios in mind when designing new features. However, they have always tried hard to ensure that each element they add is useful beyond these primary scenarios.

For example, several years ago, the C# language designers decided to add features to C# to make database access feel well integrated with the language. The resulting technology, Language Integrated Query (LINQ, described in [Chapter 10](#)), certainly supports that goal, but they achieved this without adding any direct support for data access to the language.

Instead, the design team introduced a series of quite diverse-seeming capabilities. These included better support for functional programming idioms, the ability to add new methods to existing types without resorting to inheritance, support for anonymous types, the ability to obtain an object model representing the structure of an expression, and the introduction of query syntax. The last of these has an obvious connection to data access, but the rest are harder to relate to the task at hand. Nonetheless, these can be used collectively in a way that makes certain data access tasks significantly simpler. But the features are all useful in their own right, so as well as supporting data access, they enable a much wider range of scenarios. For example, these additions made it much easier to process lists, sets, and other groups of objects, because the new features work for collections of things from any origin, not just databases.

One illustration of this philosophy of generality was a language feature that was prototyped for C# but which its designers ultimately chose not to go ahead with. The feature would have enabled you to write XML directly in your source code, embedding expressions to calculate values for certain bits of content at runtime. The prototype compiled this into code that generated the completed XML at runtime. Microsoft Research demonstrated this publicly, but this feature didn't ultimately make it into C#, although it did later ship in another .NET language, Visual Basic, which also got some specialized query features for extracting information from XML documents. Embedded XML expressions are a relatively narrow facility,

only useful when you're creating XML documents. As for querying XML documents, C# supports this functionality through its general-purpose LINQ features, without needing any XML-specific language features. XML's star has waned since this language concept was mooted, having been usurped in many cases by JSON (which may well be eclipsed by something else in years to come). Had embedded XML made it into C#, it would by now feel like an anachronistic curiosity.

The new features added in subsequent versions of C# continue in the same vein. For example, the relatively new *range* syntax (described in [Chapter 5](#)) was motivated partly by some machine learning and AI scenarios, but the feature is not limited to any particular application area. Likewise, generic math is one of the more significant new capabilities in C# 11.0, but it is enabled by some general-purpose enhancements of the type system.

C# Standards and Implementations

Before we can get going with some actual code, we need to know which implementation of C# and the runtime we are targeting. The standards body Ecma has written specifications that define language and runtime behavior (ECMA-334 and ECMA-335, respectively) for C# implementations. This has made it possible for multiple implementations of C# and the runtime to emerge. At the time of writing, there are four in widespread use: .NET, Mono, .NET Native (a forerunner of .NET Native AOT still used by applications targeting the Universal Windows Platform), and .NET Framework. Somewhat confusingly, Microsoft is behind all of these, although it didn't start out that way.

Many .NETs

The Mono project was launched in 2001 and did not originate from Microsoft. (This is why it doesn't have .NET in its name—it can use the name C# because that's what the standards call the language, but back in the pre-.NET Foundation days, the .NET brand was exclusively used by Microsoft.) Mono started out with the goal of enabling Linux desktop ap-

plication development in C#, but it went on to add support for iOS and Android. That crucial move helped Mono find its niche, because it is now mainly used to create cross-platform mobile device applications in C#. Mono also introduced support for targeting WebAssembly (also known as WASM) and includes an implementation of the CLR that can run in any standards-compliant web browser, enabling C# code to run on the client side in web applications. This is often used in conjunction with a .NET application framework called Blazor, which enables you to build HTML-based user interfaces while using C# to implement behavior. The Blazor-with-WASM combination also makes C# a viable language for working with platforms such as Electron, which use web client technologies to create cross-platform desktop applications. (Blazor doesn't require WASM—it can also work with C# code compiled normally and running on the .NET runtime; .NET's Multi-platform App UI [MAUI] exploits this to make it possible to write a single application that can run on Android, iOS, macOS, and Windows.)

Mono was open source from the start and has been supported by a variety of companies over its existence. In 2016, Microsoft acquired the company that had stewardship of Mono: Xamarin. For now, Microsoft retains Xamarin as a distinct brand, positioning it as the way to write cross-platform C# applications that can run on mobile devices. Mono's core technology has been merged into Microsoft's .NET runtime codebase. This was the endpoint of several years of convergence in which Mono gradually shared more and more in common with .NET. Initially, Mono provided its own implementations of everything: C# compiler, libraries, and the CLR. But when Microsoft released an open source version of its own compiler, the Mono tools moved over to that. Mono used to have its own complete implementation of the .NET runtime libraries, but ever since Microsoft released the first open source version of .NET, Mono has been depending increasingly on that. Today, Mono is no longer a separate .NET, being effectively one of two CLR implementations in the main .NET runtime repository, enabling support for mobile and WebAssembly runtime environments.

What about the other three implementations, all of which seem to be called .NET? There is .NET Native, a predecessor of Native AOT. It is used

in Universal Windows Platform (UWP) apps, but developers are discouraged from building new applications this way, since there are no plans for either UWP or the old .NET Native to be updated except for bug or security fixes. So in practice we have just two current, non-doomed versions: .NET Framework (Windows only, closed-source) and .NET (cross-platform, open source). However, as mentioned earlier, Microsoft is not planning to add any new features to the Windows-only .NET Framework, so this leaves .NET 8.0 as effectively the only current version.

Nonetheless, .NET Framework continues to be used because there are a handful of things it can do that .NET 8.0 cannot. .NET Framework only runs on Windows, whereas .NET 8.0 supports Windows, macOS, iOS, and Linux, and although this makes .NET Framework less widely usable, it means it can support some Windows-specific features. For example, there is a section of the .NET Framework Class Library dedicated to working with COM+ Component Services, a Windows feature for hosting components that integrate with Microsoft Transaction Server. This isn't possible on the newer, cross-platform versions of .NET because code might be running on Linux, where equivalent features either don't exist or are too different to be presented through the same .NET API.

The number of .NET-Framework-only features has dropped dramatically over the last few releases, because Microsoft has been working to enable even Windows-only applications to use the latest version of .NET. Even many Windows-specific features can be used from .NET. For example, the `System.Speech` .NET library used to be available only on .NET Framework because it provides access to Windows-specific speech recognition and synthesis functionality, but there is now a .NET version of this library. That library only works on Windows, but its availability means that application developers relying on it are now free to move from .NET Framework to .NET. The remaining .NET Framework features that have not been brought forward are those that are not used extensively enough to justify the engineering effort. COM+ support was not just a library—it had implications for how the CLR executed code, so supporting it in modern .NET would have had costs that were not justifiable for what is now a rarely used feature.

The cross-platform .NET is where most of the new development of .NET has occurred for the last few years. .NET Framework is still supported, and will be for many years to come, but Microsoft has stated that it will not get any new features, and it has been falling behind for some time. For example, Microsoft's web application framework, ASP.NET Core, dropped support for .NET Framework back in 2019. So .NET Framework's retirement, and .NET's status as the one true .NET, is the inevitable conclusion of a process that has been underway for a few years. Since many legacy projects continue to run on .NET Framework, .NET library developers often want to support both runtimes, so I will call out places where there are significant differences, but new C# applications should use .NET, not .NET Framework.

Release Cycles and Long Term Support

Microsoft currently releases new versions of C# and .NET every year, normally around November or December, but not all versions are created equal. Alternate releases get *Long Term Support* (LTS), meaning that Microsoft commits to supporting the release for at least three years. Throughout that period, the tools, libraries, and runtime will be updated regularly with security patches. .NET 8.0, released in November 2023, is an LTS release, so it will be supported until December 2026. The preceding LTS release was .NET 6.0, which was released in December 2021 and therefore remains in support until December 2024; the LTS release before that was .NET Core 3.1,^{[2](#)} which went out of support in December 2022.

What about non-LTS releases? These are supported from release but only for 18 months. For example, .NET 5.0 was supported when it was released in December 2020, but support ended in May 2022, six months after .NET 6.0 shipped (and six months *before* support for its predecessor .NET Core 3.1 ended).

It often takes a few months for the ecosystem to catch up with a new release. You might not be able to use a new version of .NET on the day of its release in practice, because your cloud platform provider might not support it yet, or there may be incompatibilities with libraries that you need to use. This significantly shortens the effective useful lifetime of non-LTS

releases, and it can leave you with an uncomfortably narrow window in which to upgrade when the next version appears. If it takes a few months for the tools, platforms, and libraries you depend on to align with the new release, you will have very little time to move on before it falls out of support. In extreme situations, this window of opportunity might not even exist: .NET Core 2.2 reached the end of its supported life before Azure Functions offered full support for either .NET Core 3.0 or 3.1, so developers who had used the non-LTS .NET Core 2.2 on Azure Functions found themselves in a situation where the latest supported version actually went backward: they had to choose between either downgrading back to .NET Core 2.1 or using an unsupported runtime in production for a few months. For this reason, some developers look at the non-LTS versions as previews—you can experimentally target new features in anticipation of using them in production once they arrive in an LTS release.

Targeting Multiple .NET Runtimes

For many years, the multiplicity of .NET runtimes, each with its own different version of the runtime libraries, presented a challenge for anyone wanting to make their C# code available to other developers. This has been improving in recent years because Microsoft made convergence a major goal with recent releases, so these days if a component targets the oldest version of .NET in support (.NET 6.0 as I write this) it will be able to run on the majority of .NET runtimes. However, it is common to want to continue to support systems that run on the old .NET Framework. This means that it will be useful to produce components that target multiple .NET runtimes for the foreseeable future.

There's a package repository for .NET components called [NuGet](#), which is where Microsoft publishes all of the .NET libraries it produces that are not built into .NET itself, and it is also where most .NET developers publish libraries they'd like to share. But which version should you build for? This is a two-dimensional question: there is the runtime implementation (.NET, .NET Framework, UWP) and also the version (for example, .NET 6.0 or .NET 8.0; .NET Framework 4.7.2 or 4.8). Many authors of popular open source packages distributed through NuGet support a plethora of versions, old and new.

Component authors often used to support multiple runtimes by building multiple variants of their libraries. When you distribute .NET libraries via NuGet, you can embed several sets of binaries in the package, each targeting different flavors of .NET. However, one major problem with this is that as new forms of .NET have appeared over the years, existing libraries wouldn't run on all newer runtimes. A component written for .NET Framework 4.0 would work on all subsequent versions of .NET Framework but not necessarily on, say, .NET 6.0. Even if the component's source code was entirely compatible with the newer runtime, you would need to compile a separate version to target that platform. And if the author of a library that you use had only provided .NET Framework binaries, it might not work on .NET. (You can tell .NET to try to use .NET Framework binaries, and it goes to some lengths to accommodate them. You might find that it does just work, but there are no guarantees.) This was bad for everyone. Various versions of .NET have come and gone over the years (such as Silverlight and several Windows Phone variants; UWP's .NET Native is still hanging in there), meaning that component authors found themselves on a treadmill of having to churn out new variants of their component. Since that relies on those authors having the inclination and time to do this work, component consumers might find that not all of the components they want to use are available on their chosen platform.

To avoid this, Microsoft introduced .NET Standard, which defines common subsets of the .NET runtime libraries' API surface area. Many of the older runtimes have gone away, but the split between .NET and .NET Framework remains, so today, .NET Standard 2.0 is likely to be the best choice for component authors wishing to support a wide range of platforms, because all recently released versions of .NET support it, and it provides access to a very broad set of features. If you don't need to support .NET Framework, it would make more sense to target .NET 6.0 or .NET 8.0 instead. [Chapter 12](#) describes some of the considerations around .NET Standard in more detail.

NOTE

When a version of .NET goes out of support, this does not deprecate components that target it. .NET supports using components built for older versions, so if you find a component on NuGet that targets .NET 5.0, you can use it on .NET 8.0. It would be wise to check whether such a component is still under active development, but old targets don't necessarily imply that a component is out of date. Sometimes component authors choose to help out people who are running systems on unsupported runtimes.

Microsoft provides more than just a language and the various runtimes with its associated class libraries. There are also development environments that can help you write, test, debug, and maintain your code.

Visual Studio, Visual Studio Code, and JetBrains Rider

Microsoft offers two desktop development environments: Visual Studio Code and Visual Studio. Both provide the basic features—such as a text editor, build tools, and a debugger—but Visual Studio provides the most extensive support for developing C# applications, whether those applications will run on Windows or other platforms. It has been around the longest—for as long as C#—so it comes from the pre-open source days and continues to be a closed-source product. The various editions available range from free to eye-wateringly expensive. Microsoft is not the only option: the developer productivity company JetBrains sells a fully fledged .NET IDE called Rider, which runs on Windows, Linux, and macOS.

Visual Studio is an Integrated Development Environment (IDE), so it takes an “everything included” approach. In addition to a fully featured text editor, it offers visual editing tools for UIs. There is deep integration with source control systems such as Git and with online systems such as GitHub and Microsoft's Azure DevOps system that provide source repositories, issue tracking, and other Application Lifecycle Management (ALM) features. Visual Studio offers built-in performance monitoring and diag-

nostic tools. It has various features for working with applications developed for and deployed to Microsoft's Azure cloud platform. It has the most extensive set of refactoring features out of the three Microsoft environments described here. Note that this version of Visual Studio runs only on Windows.

The JetBrains Rider IDE is a single product that runs on Windows, macOS, and Linux. It is more focused than Visual Studio, in that it was designed purely to support .NET application development. (Visual Studio also supports C++.) It has a similar “everything included” approach, and it offers a particularly powerful range of refactoring tools.

Visual Studio Code (often shortened to VS Code) was first released in 2015. It is open source and cross platform, supporting Linux as well as Windows and Mac. It is based on the Electron platform and is written predominantly in TypeScript. (This means that VS Code really is the same program on all operating systems.) VS Code is a more lightweight product than Visual Studio: a basic installation of VS Code has little more than text editing support. However, as you open up files, it will discover downloadable extensions that, if you choose to install them, can add support for C#, F#, TypeScript, PowerShell, Python, and a wide range of other languages. (The extension mechanism is open, so anyone who wants to can publish an extension.) So although in its initial form it is less of an IDE and more like a simple text editor, its extensibility model makes it pretty powerful. The wide range of extensions has led to VS Code becoming remarkably popular outside of the world of Microsoft languages, and this in turn has encouraged a virtuous cycle of even greater growth in the range of extensions.

Visual Studio and JetBrains Rider offer the most straightforward path to getting started in C#—you don't need to install any extensions or modify any configuration to get up and running. However, Visual Studio Code is available to a wider audience, so I'll be using that in the quick introduction to working with C# that follows. The same basic concepts apply to all environments, though, so if you will be using Visual Studio or Rider, most of what I describe here still applies.

TIP

You can download [Visual Studio Code for free](#). You will also need to [install the .NET SDK](#). If you are using Windows and would prefer to use Visual Studio, you can download the free version of Visual Studio, called [Visual Studio Community](#). This will install the .NET SDK for you, as long as you select at least one .NET *workload* during installation.

Any nontrivial C# application will have multiple source code files, and these will belong to a *project*. Each project builds a single output, or *target*. The build target might be as simple as a single file—a C# project could produce an executable file or a library, for example—but some projects produce more complicated outputs. For instance, some project types build websites. A website will normally contain multiple files, but collectively, these files represent a single entity: one website. Each project's output will be deployed as a unit, even if it consists of multiple files.

Executables typically have a *.exe* file extension in Windows, while libraries use *.dll* (historically short for *dynamic link library*). With .NET, however, all code goes into *.dll* files, even on macOS and Linux. The SDK can also generate a *host* executable (with a *.exe* extension on Windows), but this just starts the runtime and then loads the *.dll* containing the main compiled output. (It's slightly different if you target .NET Framework: that compiles the application directly into a self-bootstrapping *.exe* with no separate *.dll*.) In any case, the only difference between the main compiled output of an application and a library is that the former specifies an application entry point. Both file types can export features to be consumed by other components. These are both examples of *assemblies*, the subject of [Chapter 12](#). (If you use Native AOT you will end up with a *.exe* on Windows and a similarly executable binary on other platforms, but Native AOT essentially works as an extra final step: it takes the various *.dll* files produced by the normal .NET build process and compiles these into a single native executable.)

C# project files have a *.csproj* extension, and if you examine these files with a text editor, you'll find that they contain XML. A *.csproj* file describes the contents of the project and configures how it should be built.

The Visual Studio and Rider IDEs know how to process these, and so do the .NET extensions for VS Code. They are also understood by various command-line build utilities such as the `dotnet` command-line tool installed by the .NET SDK, and also Microsoft's older MSBuild tool. (MSBuild supports numerous languages and targets, not just .NET. In fact, when you build a C# project with the .NET SDK's `dotnet build` command, it is effectively a wrapper around MSBuild.)

You will often want to work with groups of projects. For example, it is good practice to write tests for your code, but most test code does not need to be deployed as part of the application, so you would typically put automated tests into separate projects. And you may want to split up your code for other reasons. Perhaps the system you're building has a desktop application and a website, and you have common code you'd like to use in both applications. In this case, you'd need one project that builds a library containing the common code, another producing the desktop application executable, another to build the website, and three more projects containing the tests for each of the main projects.

The build tools and IDEs that understand .NET help you work with multiple related projects through what they call a *solution*. A solution is a file with a `.sln` extension, defining a collection of projects. While the projects in a solution are usually related, they don't have to be.

If you're using Visual Studio, be aware that it requires projects to belong to a solution, even if you have only one project. Visual Studio Code is happy to open a single project if you want, but its .NET extensions also recognize solutions.

A project can belong to more than one solution. In a large codebase, it's common to have multiple `.sln` files with different combinations of projects. You would typically have a main solution that contains every single project, but not all developers will want to work with all the code all of the time. Someone working on the desktop application in our hypothetical example will also want the shared library but probably has no interest in loading the web project.

I'll show how to create a new project, open it in Visual Studio Code, and run it. I'll then walk through the various features of a new C# project as an introduction to the language. I'll also show how to add a unit test project and how to create a solution containing both projects.

Anatomy of a Simple Program

Once you've installed the .NET 8.0 SDK either directly or by installing an IDE, you can create a new .NET program. Start by creating a new directory called *HelloWorld* on your computer to hold the code. Open up a command prompt and ensure that its current directory is set to that, and then run this command:

```
dotnet new console
```

This makes a new C# console application by creating two files. It creates a project file with a name based on the parent directory: *HelloWorld.csproj* in this case. And there will be a *Program.cs* file containing the code. If you open that file up in a text editor, you'll see it's pretty simple, as [Example 1-1](#) shows.

Example 1-1. Our first program

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("Hello, World!");
```

You can compile and run this program with the following command:

```
dotnet run
```

As you've probably already guessed, this will display the text `Hello, World!`, the traditional behavior for the opening example in any programming book.

Over half of this example is just a comment. The second line here is all you need, with the first just showing a link explaining the “new” style this project uses. It’s not all that new anymore—it came in with the .NET 6.0 SDK—but there was a significant change in the language, and the .NET SDK authors felt it necessary to provide an explanation. The last few versions of C# have added various features intended to reduce the amount of *boilerplate*. Boilerplate is the name used to describe code that needs to be present to satisfy certain rules or conventions but that looks more or less the same in any project. For example, C# requires code to be defined inside a *method*, and a method must always be defined inside a *type*. You can see evidence of these rules in [Example 1-1](#). To produce output, it relies on the .NET runtime’s ability to display text, which is embodied in a method called `WriteLine`. But we don’t just say `WriteLine` because C# methods always belong to types, which is why the code qualifies this as `Console.WriteLine`.

Any C# code that we write is subject to the rules, so our code that invokes the `Console.WriteLine` method must itself live inside a method inside a type. And in the majority of C# code, this would be explicit: in most cases, you’ll see something a bit more like [Example 1-2](#).

Example 1-2. “Hello, World!” with visible boilerplate

```
using System;

namespace HelloWorld;

internal class Program
{
    private static void Main(string[] args)
    {
        Console.WriteLine("Hello, World!");
    }
}
```

There’s still only one line here that defines the behavior of the application, and it’s the same as in [Example 1-1](#). The obvious advantage of the first example is that it lets us focus on what our program actually does, al-

though the downside is that quite a lot of what's going on becomes invisible. With the explicit style in [Example 1-2](#), nothing is hidden. [Example 1-1](#) uses the *top-level statement* style, but the compiler still puts the code in a method defined inside a type called `Program`; it's just that you can't see that from the code. With [Example 1-2](#), the method and type are clearly visible.

The C# boilerplate reduction feature that enables us to dive straight in with the code is just for the program entry point. When you're writing the code you want to execute whenever your program starts, you don't need to define a containing class or method. But a program has only one entry point, and for everything else, you still need to spell it out. So in practice, most C# code looks more like [Example 1-2](#) than [Example 1-1](#), and with older codebases even the program entry point will use this explicit style.

Since real projects involve multiple files, and usually multiple projects, let's move on to a slightly more realistic example. I'm going to create a program that calculates the average (the arithmetic mean, to be precise) of some numbers. I will also create a second project that will automatically test our first one. Since I've got two projects, this time I'll need a solution. I'll create a new directory called *Averages*. If you're following along, it doesn't matter where it goes, although it's a good idea *not* to put it inside the *HelloWorld* project's directory. I'll open a command prompt in the *Averages* directory and run this command:

```
dotnet new sln
```

This will create a new solution file named *Averages.sln*. (By default, `dotnet new` usually names new projects and solutions after their containing directories, although you can specify other names.) Now I will add the two projects I need with these two commands:

```
dotnet new console -o Averages
dotnet new mstest -o Averages.Tests
```

The `-o` option here (short for `output`) indicates that I want the tool to create a new subdirectory for each of these new projects—when you have multiple projects, each needs its own directory.

I now need to add these to the solution:

```
dotnet sln add ./Averages/Averages.csproj  
dotnet sln add ./Averages.Tests/Averages.Tests.csproj
```

I'm going to use that second project to define some tests that will check the code in the first project (which is why I specified a project type of `mstest`—this project will use Microsoft's unit test framework). This means that the second project will need access to the code in the first project. To enable that, I run this command:

```
dotnet add ./Averages.Tests/Averages.Tests.csproj reference  
./Averages/Averages.csproj
```

(I've split this over two lines to make it fit, but it needs to be run as a single command.) Finally, to edit the project, I can launch VS Code in the current directory with this command:

```
code .
```

If you're following along, and if this is the first time you've run VS Code, it will ask you to make some decisions, such as choosing a color scheme. It might also ask you whether you trust the folder location, in which case you should tell it that you do. You might be tempted to ignore its questions, but one of the things it might offer to do at this point is install extensions for language support. People use VS Code with all sorts of languages, and the installer makes no assumptions about which you will be using, so you have to install an extension to get C# support. If you follow VS Code's instructions to browse for language extensions, it will offer Microsoft's C# Dev Kit extension. Don't panic if VS Code does not offer to do this. The automatic extension suggestion behavior has changed from time to time, and it might have changed again after I wrote this. You

might need to open one of the `.cs` files before it shows a prompt similar to the one in [Figure 1-1](#).

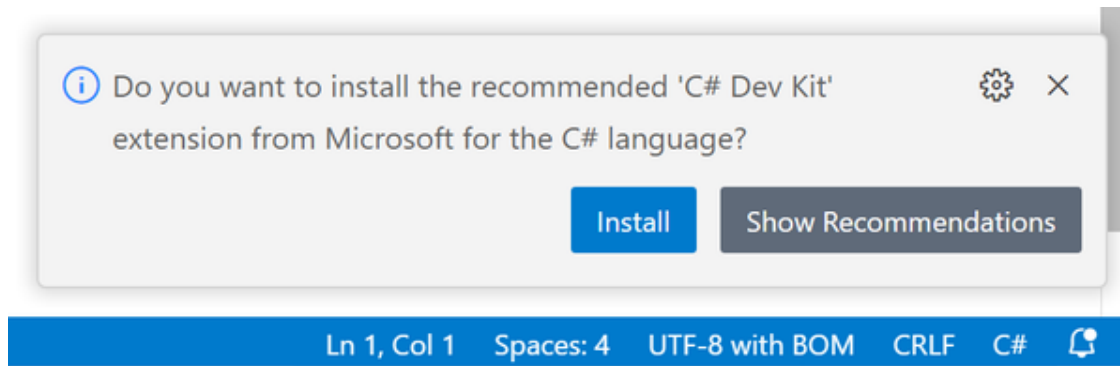


Figure 1-1. Visual Studio Code's extension suggestion prompt

If you don't see this, perhaps you already had the C# Dev Kit extension installed. To find out, click the Extensions icon on the bar on the lefthand side. It's the one shown at the bottom left of [Figure 1-2](#), with four squares. If you've opened VS code in a directory with a `.csproj` file in it, the C# Dev Kit extension should appear in the INSTALLED section if you've already installed it, and in the RECOMMENDED section if you don't have it yet.

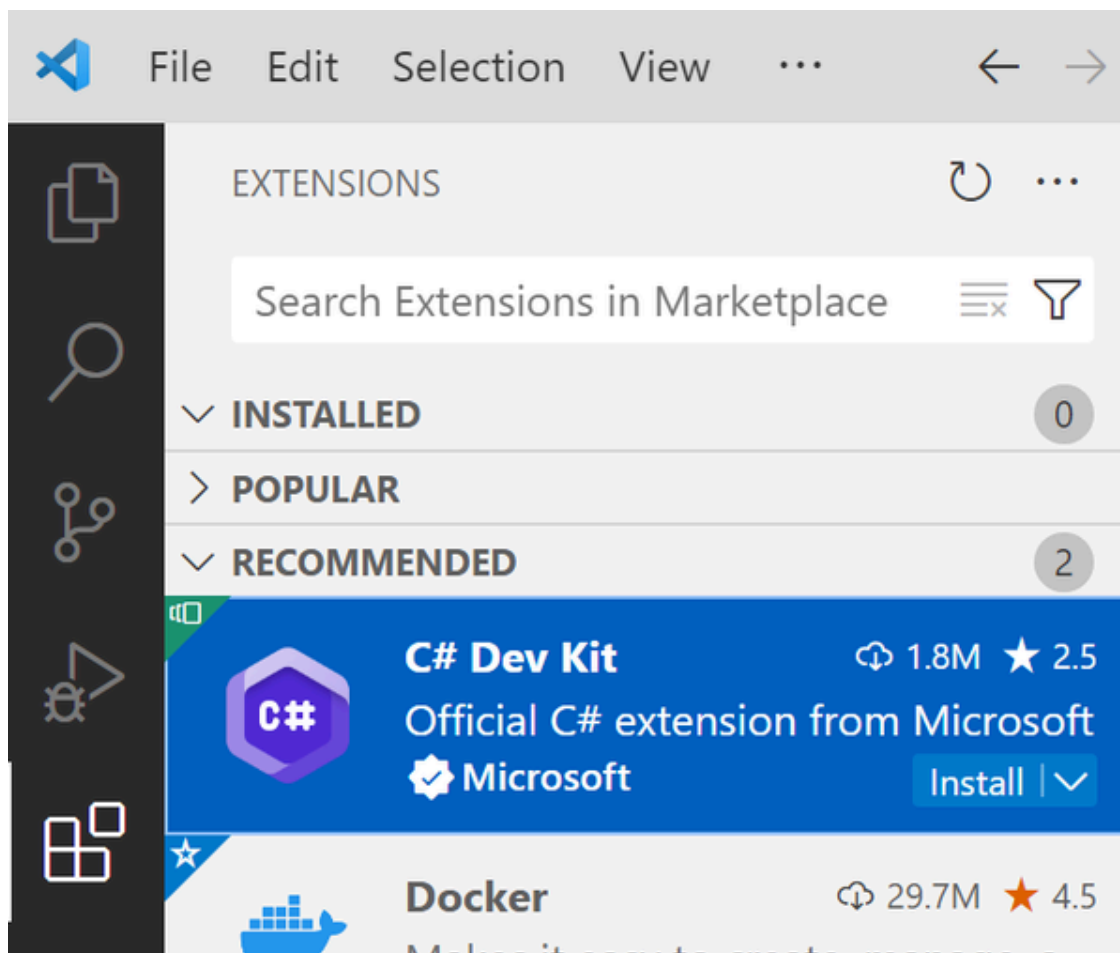


Figure 1-2. Visual Studio Code's C# Dev Kit extension

If you still don't see it, type **C#** into the search text box at the top. A few results will appear, so if you're following along, make sure you get the right one. If you click the search result, it will show more detailed information, which should show its full name as "C# Dev Kit" with a subtitle of "Official C# extension from Microsoft" and it will show "Microsoft" as the publisher. Click the Install button to install the extension.

It might take a few minutes to download and install the C# Dev Kit extension, but once that's done, at the bottom left of the window the status bar should look similar to [Figure 1-3](#), showing it has found the two projects we created.



Figure 1-3. Visual Studio Code's status bar

The C# Dev Kit extension will inspect all of the source code in all of the projects in the solution. Obviously there's not much in these yet, but it will continue to analyze code as I type, enabling it to identify problems and make helpful suggestions.

I can take a look at the code by switching to the Explorer view, using the button at the top of the toolbar on the left. As [Figure 1-4](#) shows, it displays the directories and files. I've expanded the *Averages.Tests* directory and have selected its *UnitTest1.cs* file.

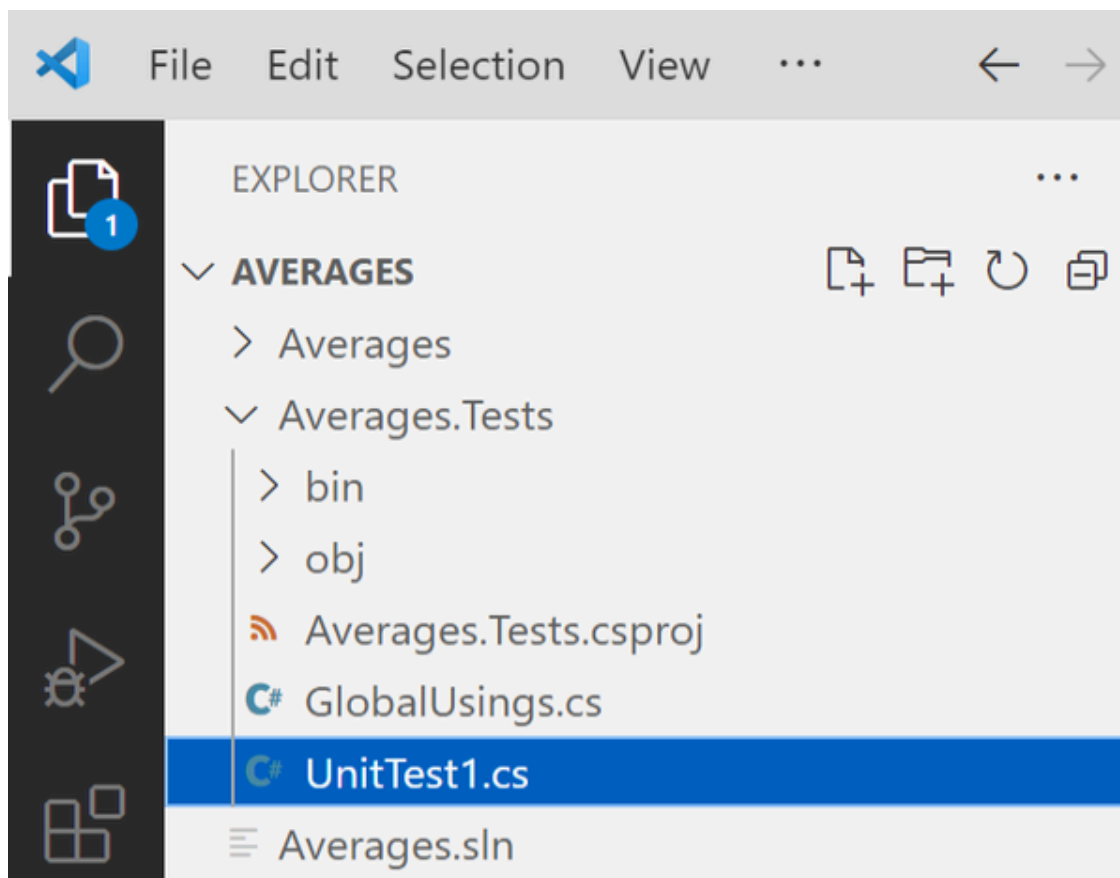


Figure 1-4. Visual Studio Code's Explorer

TIP

If you single-click a file in the Explorer panel, VS Code shows it in a *preview tab*, meaning that it won't stay open for long: as soon as you click some other file, that displaces the one you had open before. This is designed to avoid ending up with hundreds of open tabs, but if you're working back and forth across two files, this can be annoying. You can avoid this by double-clicking the file when you open it—that opens a nonpreview tab, which will remain open until you deliberately close it. If you already have a file open in a preview tab, you can double-click the filename to turn it into an ordinary tab. VS Code shows the filename in italics in preview tabs, and you'll see it change to nonitalic when you double-click.

You might be wondering why I expanded the *Averages.Tests* directory. The purpose of this test project will be to ensure that the main project does what it's supposed to. With this example, I'll be following the engineering practice of defining tests that embody my requirements before writing the code being tested (sometimes known as *test driven development* or TDD). That means starting with the test project.

Writing a Unit Test

When I ran the command to create this project earlier, I specified a project type of `mstest`. This project template has provided me with a test class to get me started, in a file called *UnitTest1.cs*. I want to pick a more informative name. There are various schools of thought as to how you should structure your unit tests. Some developers advocate one test class for each class you wish to test, but I like the style where you write a class for each *scenario* in which you want to test a particular class, with one method for each of the things that should be true about your code in that scenario. This program will only have one behavior: it will calculate the arithmetic mean of its inputs. So I'll rename the *UnitTest1.cs* source file to *WhenCalculatingAverages.cs*. (You can rename a file by right-clicking it in VS Code's Explorer panel and selecting the Rename entry.) This test should verify that we get the expected results for a few representative inputs. [Example 1-3](#) shows a complete source file that does this; there are two tests here, shown in bold.

Example 1-3. A unit test class for our first program

```
using Microsoft.VisualStudio.TestTools.UnitTesting;

namespace Averages.Tests;

[TestClass]
public class WhenCalculatingAverages
{
    [TestMethod]
    public void SingleInputShouldProduceSameValueAsResult()
    {
        string[] inputs = { "1" };
        double result = AverageCalculator.ArithmeticMean(inputs);
        Assert.AreEqual(1.0, result, 1E-14);
    }

    [TestMethod]
    public void MultipleInputsShouldProduceAverageAsResult()
    {
        string[] inputs = { "1", "2", "3" };
    }
}
```



```
        double result = AverageCalculator.ArithmeticMean(inputs);
        Assert.AreEqual(2.0, result, 1E-14);
    }
}
```

I will explain each of the features in this file once I've shown the program itself. For now, the most interesting parts of this example are the two methods. First, we have the

`SingleInputShouldProduceSameValueAsResult` method, which checks that our program correctly handles the case where there is a single input. The first line inside this method describes the input—a single number. (Slightly surprisingly, this test represents the numbers as strings. This is because our inputs will ultimately come as command-line arguments, so our test needs to reflect that.) The second line executes the code under test (which I've not actually written yet). And the third line states that the calculated average should be equal to the one and only input. If it's not, this test will report a failure. The second method, `MultipleInputsShouldProduceAverageAsResult`, checks a slightly more complex case, in which there are three inputs, but has the same basic shape as the first.

NOTE

This code uses C#'s `double` type, a double-precision floating-point number, to be able to represent results that are not whole numbers. I'll be describing C#'s built-in data types in more detail in the next chapter, but be aware that as with most programming languages, floating-point arithmetic in C# has limited precision. The `Assert.AreEqual` method I'm using to check the results here takes this into account and lets me specify maximum tolerance for imprecision. The final argument of `1E-14` in each case denotes the number 1 divided by 10 raised to the power of 14, so these tests are stating that they require the answer to be correct to 14 decimal places.

Let's focus on one particular line from these tests: the one that runs the code I want to test. [Example 1-4](#) shows the relevant line from [Example 1-3](#). This is how you invoke a method that returns a result in C#. This line starts by declaring a variable to hold the result. (The text `double` indi-

cates the data type, and `result` is the variable's name.) All methods in C# need to be defined inside a type, so just as we saw earlier with the `Console.WriteLine` example, we have the same form here: a type name, then a period, then a method name. And then, in parentheses, the input to the method.

Example 1-4. Calling a method

```
double result = AverageCalculator.ArithmeticMean(inputs);
```

If you are following along by typing the code in as you read, then if you look at the two places this line of code appears (once in each test method), you might notice that VS Code has drawn a squiggly line underneath `AverageCalculator`. Hovering the mouse over this kind of squiggly shows an error message, as [Figure 1-5](#) shows.

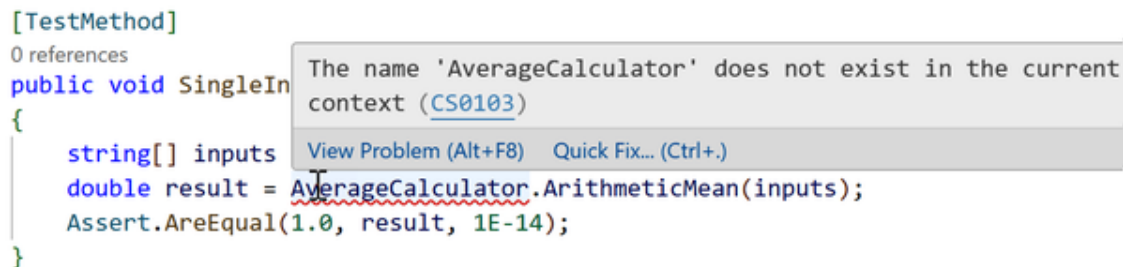


Figure 1-5. An unrecognized type

This is telling us something we already knew: I haven't yet written the code that this test aims to test. Let's fix that. I need to add a new file, which I can do in VS Code's Explorer view by clicking the *Averages* directory and then, with that selected, clicking the leftmost button on the toolbar near the top of the Explorer. [Figure 1-6](#) shows that when you hover the mouse over this button, it shows a tool tip confirming its purpose. After clicking it, I can type in *AverageCalculator.cs* as the name for the new file.

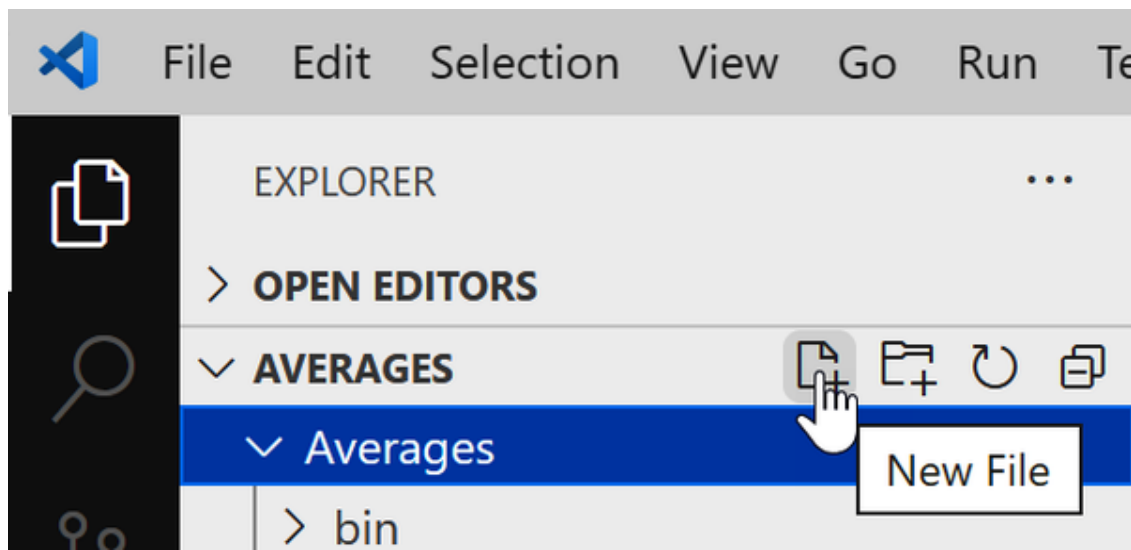


Figure 1-6. Adding a new file

VS Code will create a new, empty file. I'll add the smallest amount of code I can to fix the error reported in [Figure 1-5](#). [Example 1-5](#) will satisfy the C# compiler. It's not complete yet—it doesn't perform the necessary calculations, but we'll come to that.

Example 1-5. A simple class

```
namespace Averages;

public static class AverageCalculator
{
    public static double ArithmeticMean(string[] args)
    {
        return 1.0;
    }
}
```

You can now build the code. If you look at the bottom of VS Code's Explorer, you should see a Solution Explorer section. If you expand this, it will show the solution and its two projects. You can right-click the solution and select Build, as [Figure 1-7](#) shows. Alternatively, you can just type `dotnet build` at the command line. Or you can use the Ctrl-Shift-B shortcut. The first time you use that shortcut, it may ask you to confirm that you want to use the `dotnet build` command to perform the build.

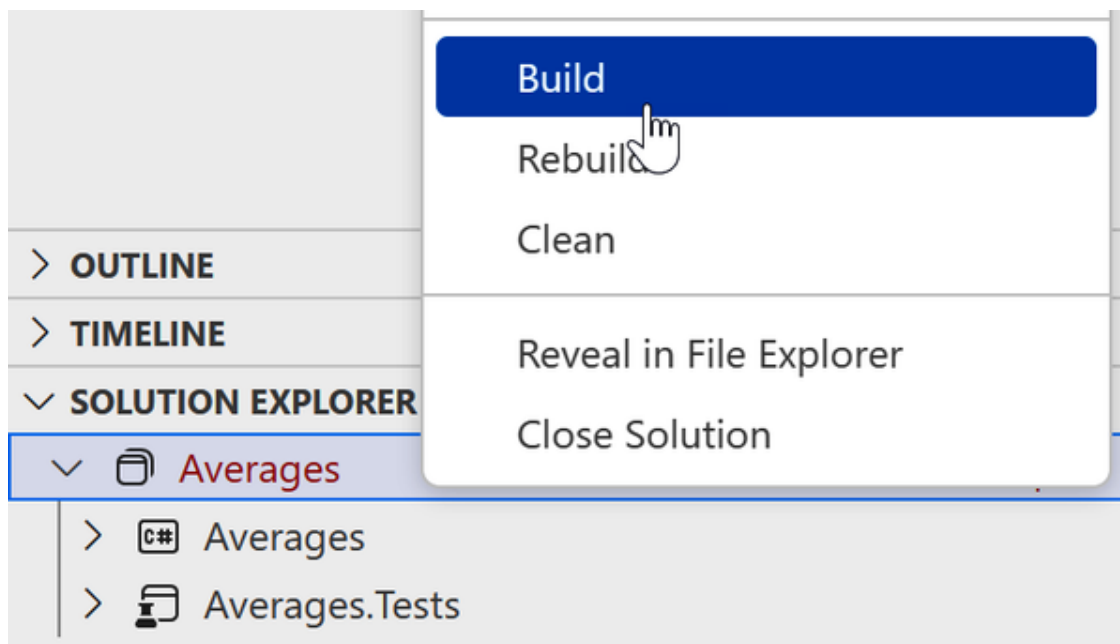


Figure 1-7. Building the solution

Once the build is complete, you should see a flask icon on the button bar on the left of the VS Code window, as [Figure 1-8](#) shows. If you click this, it will show the Testing panel, enabling you to run tests and see the results.

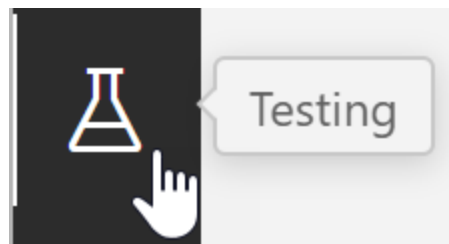


Figure 1-8. The Testing button

The Testing panel shows the tests hierarchically, first grouped by project and then by namespace. (Since your test project is called `Averages.Tests`, and all its tests are in a namespace called `Averages.Tests`, you'll see that appear twice, as [Figure 1-9](#) shows.) At the top of the Testing panel is a row of buttons. One shows a couple of triangles, one on top of the other. If you click this it will run all of the tests.

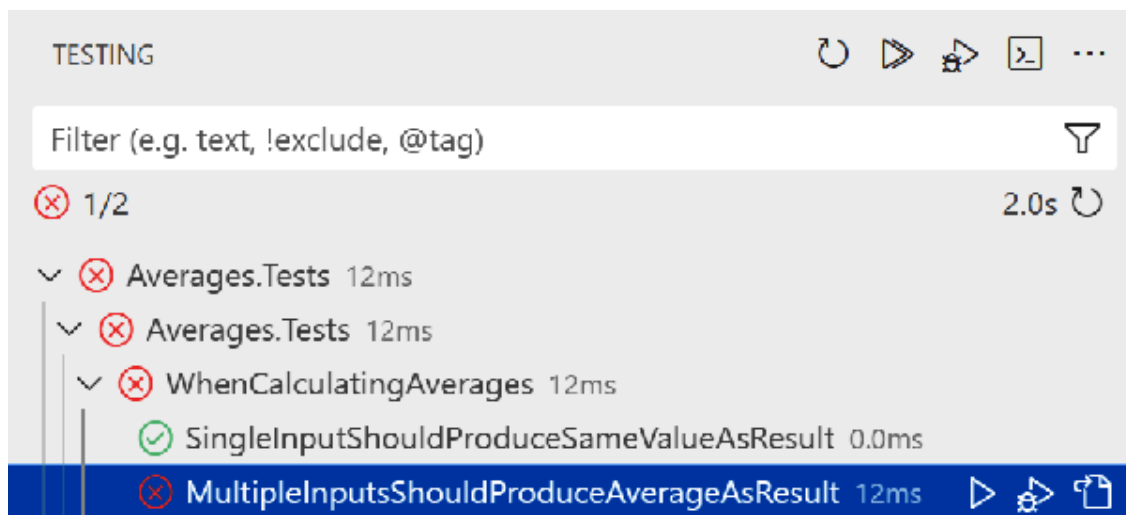


Figure 1-9. Test results

Since we've not finished writing the library yet, one of these tests will fail. As [Figure 1-9](#) shows, this is indicated by showing a cross next to the test. If you click the failing test, VS Code will show you the point in the code at which the test failed, along with details including the failure message and a stack trace, as [Figure 1-10](#) shows.

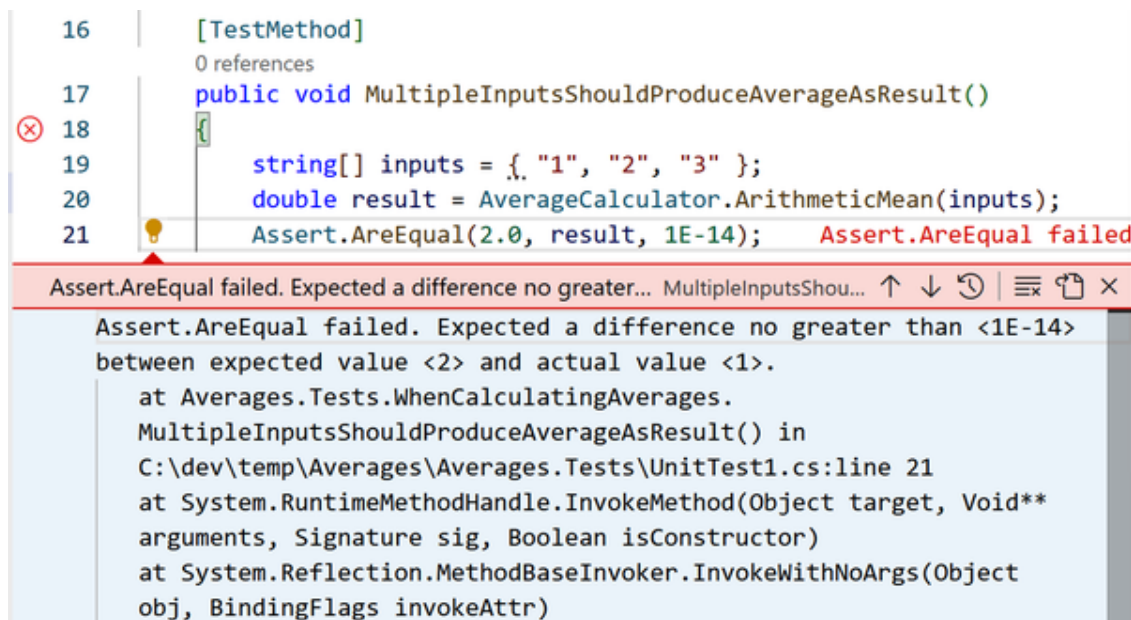


Figure 1-10. A failing test

You can alternatively run tests with the command `dotnet test`. It shows the same information as Visual Studio Code did, but just as normal console output:

```
Failed MultipleInputsShouldProduceAverageAsResult [291 ms]
Error Message:
  Assert.AreEqual failed. Expected a difference no greater than <1E-14>
```

```
between expected value <2> and actual value <1>.
Stack Trace:
    at Averages.Tests.WhenCalculatingAverages.
MultipleInputsShouldProduceAverageAsResult() in
C:\book\Averages\Averages.Tests\WhenCalculatingAverages.cs:line 21
    at System.RuntimeMethodHandle.InvokeMethod(Object target, Void** arguments,
Signature sig, Boolean isConstructor)
    at System.Reflection.MethodBaseInvoker.InvokeWithNoArgs(Object obj,
BindingFlags invokeAttr)

Failed! - Failed:      1, Passed:      1, Skipped:      0, Total:      2,
Duration: 364 ms - Averages.Tests.dll (net6.0)
```

As expected, we get failures because I've not written a proper implementation yet. But first, I want to explain each element of [Example 1-5](#) in turn, as it provides a useful introduction to some important elements of C# syntax and structure. The very first thing in this file is a *namespace declaration*.

Namespaces

Namespaces bring order and structure to what would otherwise be a horrible mess. The .NET runtime libraries contain thousands of types, and there are many more out there in NuGet packages both from Microsoft and third parties, not to mention the classes you will write yourself. There are two problems that can occur when dealing with this many named entities. First, it becomes hard to guarantee uniqueness. Second, it can become challenging to discover the API you need; unless you know or can guess the right name, it's difficult to find what you need from an unstructured list of tens of thousands of things. Namespaces solve both of these problems.

Most .NET types are defined in a namespace. There are certain conventions for namespaces that you'll see a lot. For example, types in .NET's runtime libraries are in namespaces that start with `System`. Additionally, Microsoft has made a wide range of useful libraries available that are not a core part of .NET, and these usually begin with `Microsoft`, or, if they are for use only with some particular technology, they might be named

for that. (For example, there are libraries for using Microsoft's Azure cloud platform that define types in namespaces that start with `Azure`.) Libraries from other vendors tend to start with the company name or a product name, while open source libraries often use their project name. You are not forced to put your own types into namespaces, but it's recommended that you do. C# does not treat `System` as a special namespace, so nothing's stopping you from using that for your own types, but unless you're writing a contribution to the .NET runtime libraries that you will be submitting as a pull request to [the .NET runtime source repository](#), then it's a bad idea because it will tend to confuse other developers. You should pick something more distinctive for your own code, such as your company or project name. As you can see from the first line of [Example 1-5](#), I've chosen to define our `AverageCalculator` class inside a namespace called `Averages`, matching our project name.

The style of namespace declaration in [Example 1-5](#) is relatively new, so you are likely to come across the older, slightly more verbose style shown in [Example 1-6](#). The difference is that the namespace declaration is followed by braces (`{ }`), and its effect applies only to the contents of those braces. This makes it possible for a single file to contain multiple namespace declarations. But in practice, the overwhelming majority of C# files contain exactly one namespace declaration. With the old syntax, this means that the majority of the contents of each file has to sit inside a pair of braces, indented by one tab stop. The newer style shown in [Example 1-5](#) applies to all types declared in the file without needing to wrap them explicitly, reducing unproductive clutter in our source files.

Example 1-6. Explicitly scoped namespace declaration

```
namespace Averages
{
    public static class AverageCalculator
    {
        ...as before...
    }
}
```


The namespace usually gives a clue as to the purpose of a type. For example, all the runtime library types that relate to file handling can be found in the `System.IO` namespace, while those concerned with networking are under `System.Net`. Namespaces can form a hierarchy. The runtime libraries' `System` namespace contains types and also other namespaces, such as `System.Net`, and these often contain yet more namespaces, such as `System.Net.Sockets` and `System.Net.Mail`. These examples show that namespaces act as a sort of description, which can help you navigate the library. If you were looking for regular expression handling, for example, you might look through the available namespaces and notice the `System.Text` namespace. Looking in there, you'd find a `System.Text.RegularExpressions` namespace, at which point you'd be pretty confident that you were looking in the right place.

Namespaces also provide a way to ensure uniqueness. The namespace in which a type is defined is part of that type's full name. This lets libraries use short, simple names for things. For example, the regular expression API includes a `Capture` class that represents the results from a regular expression capture. If you are working on software that deals with images, the term *capture* is commonly used to mean the acquisition of some image data, and you might feel that `Capture` is the most descriptive name for a class in your own code. It would be annoying to have to pick a different name just because the best one is already taken, particularly if your image acquisition code has no use for regular expressions, meaning that you weren't even planning to use the existing `Capture` type.

But in fact, it's fine. Both types can be called `Capture`, and they will still have different names. The full name of the regular expression `Capture` class is effectively `System.Text.RegularExpressions.Capture`, and likewise, your class's full name would include its containing namespace (for example, `SpiffingSoftworks.Imaging.Capture`).

If you really want to, you can write the fully qualified name of a type every time you use it, but most developers don't want to do anything quite so tedious, which is where the `using` directives you can see at the start of Examples [1-2](#) and [1-3](#) come in. It's common to see a list of directives at the top of each source file, stating the namespaces of the types that file in-

tends to use. You will normally edit this list to match your file's requirements. In this example, the `dotnet` command-line tool added `using Microsoft.VisualStudio.TestTools.UnitTesting;` when it created the test project. You'll see different sets in different contexts. If you add a class representing a UI element, for example, Visual Studio would include various UI-related namespaces in the list.

If a project makes heavy use of a particular namespace, we can avoid having to put the same `using` directive in every single source file by writing a *global using directive*. If we put the `global` keyword in front of the directive, as [Example 1-7](#) does, the directive applies to all files in a project. The .NET SDK takes this a step further, by generating a hidden file in your project with a set of these `global using` directives to ensure that commonly used namespaces such as `System` and `System.Collections.Generic` are available. (The exact set of namespaces added as implicit global imports varies by project type—web projects get a few extra, for example. If you're wondering why unit test projects don't already do what [Example 1-7](#) does, it's because the .NET SDK doesn't have a specific project type for test projects—the `mstest` template we told `dotnet new` to use just creates an ordinary class library project with a reference to the unit test library packages.)

Example 1-7. A `global using` directive

```
global using Microsoft.VisualStudio.TestTools.UnitTesting;
```

With `using` declarations like these (either per-file or global) in place, you can just use the short, unqualified name for a class. The line of code that enables [Example 1-1](#) to do its job uses the `System.Console` class, but because the SDK adds an implicit `global using` directive for the `System` namespace, it can refer to it as just `Console`.

Namespaces and component names

Earlier, I used the `dotnet` CLI to add a reference from our `Averages.Tests` project to our `Averages` project. You might think that references are redundant—can't the compiler work out which external li-

libraries we are using from the namespaces? It could if there was a direct correspondence between namespaces and either libraries or packages, but there isn't.

Library names sometimes align with namespaces—the popular `Newtonsoft.Json` NuGet package contains a *Newtonsoft.Json.dll* file that contains classes in the `Newtonsoft.Json` namespace, for example. But this is an optional convention, and often there's no such connection—the .NET runtime libraries include a *System.Private.CoreLib.dll* file, but there is no `System.Private.CoreLib` namespace. So it is necessary to tell the compiler which libraries your project depends on, and also which namespaces it uses. We will look at the nature and structure of library files in more detail in [Chapter 12](#).

Resolving ambiguity

Even with namespaces, there's potential for ambiguity. A single source file might use two namespaces that both happen to define a class of the same name. If you want to use that class, then you will need to be explicit, referring to it by its full name. If you need to use such classes a lot in the file, you can still save yourself some typing: you only need to use the full name once because you can define a `using alias`. [Example 1-8](#) defines two aliases to resolve a clash that I've run into a few times: .NET's desktop UI framework, the Windows Presentation Foundation (WPF), defines a `Path` class for working with Bézier curves, polygons, and other shapes, but there's also a `Path` class for working with filesystem paths, and you might want to use both types together to produce a graphical representation of the contents of a file. Just adding `using` directives for both namespaces would make the simple name `Path` ambiguous if unqualified. But as [Example 1-8](#) shows, you can define distinctive aliases for each.

Example 1-8. Resolving ambiguity with aliases

```
using System.IO;
using System.Windows.Shapes;
```

```
using IoPath = System.IO.Path;  
using WpfPath = System.Windows.Shapes.Path;
```

With these aliases in place, you can use `IoPath` as a synonym for the file-related `Path` class, and `WpfPath` for the graphical one.

By the way, you can refer to types in your own namespace without qualification, without needing a `using` directive. That's why the test code in [Example 1-3](#) doesn't have a `using Averages;` directive. However, you might be wondering how this works, since the test code declares a different namespace, `Averages.Tests`. To understand this, we need to look at namespace nesting.

Nested namespaces

As you've already seen, the .NET runtime libraries nest their namespaces, sometimes quite extensively, and you will often want to do the same. There are two ways you can do this. You can nest namespace declarations, as [Example 1-9](#) shows.

Example 1-9. Nesting namespace declarations

```
namespace MyApp  
{  
    namespace Storage  
    {  
        ...  
    }  
}
```

Alternatively, you can just specify the full namespace in a single declaration, as [Example 1-10](#) shows. This is the more commonly used style. This single-declaration style works with either the newer kind of declaration shown in [Example 1-10](#) or with the older style using braces.

Example 1-10. Nested namespace with a single declaration

```
namespace MyApp.Storage;
```

Any code you write in a nested namespace will be able to use types not just from that namespace but also from its containing namespaces without qualification. Code in Examples [1-9](#) or [1-10](#) would not need explicit qualification or `using` directives to use types either in the `MyApp.Storage` namespace or the `MyApp` namespace. This is why in [Example 1-3](#) I didn't need to add a `using Averages;` directive to be able to access the `AverageCalculator` in the `Averages` namespace: the test was declared in the `Averages.Tests` namespace, and since that is nested in the `Averages` namespace, the code automatically has access to that outer namespace.

When you define nested namespaces, the convention is to create a matching directory hierarchy. Some tools expect this. Although VS Code doesn't currently have any particular expectations here, Visual Studio does follow this convention. If your project is called `MyApp`, it will put new classes in the `MyApp` namespace when you add them to the project. But if you create a new directory in the project called, say, *Storage*, Visual Studio will put any new classes you create in that directory into the `MyApp.Storage` namespace. Again, you're not required to keep this—Visual Studio just adds a namespace declaration when creating the file, and you're free to change it. The compiler does not need the namespace to match your directory hierarchy. But since the convention is supported by various tools, including Visual Studio, life will be easier if you follow it.

Classes

After the namespace declaration, our *AverageCalculator.cs* file defines a class. [Example 1-11](#) shows this part of the file. This starts with the `public` keyword, which enables this class to be accessed by other components. Next is the `static` keyword, which indicates that this class is not meant to be instantiated—it offers only class-level operations and no per-instance features. Then comes the `class` keyword followed by the name, and of course the full name of the type is effectively `Averages.AverageCalculator`, because of the namespace declaration. As you can see, C# uses braces (`{}`) to delimit all sorts of things—we already saw this in the older (but still widely used) namespace declaration syntax,

and here you can see the same thing with the class, as well as the method it contains.

Example 1-11. A class with a method

```
public static class AverageCalculator
{
    public static double ArithmeticMean(string[] args)
    {
        return 1.0;
    }
}
```

Classes are C#'s mechanism for defining entities that combine state and behavior, a common object-oriented idiom. But this class contains nothing more than a single method. C# does not support global methods—all code has to be written as a member of some type. So this particular class isn't very interesting—its only job is to act as the container for the method that will do the actual work. We'll see some more interesting uses for classes in [Chapter 3](#).

As with the class, I've marked the method as `public` to enable access from other components. I've also declared this to be a *static method*, meaning that it is not necessary to create an instance of the containing type (`AverageCalculator`, in this case) in order to invoke the method. The `double` keyword that follows indicates that the type of data this method returns is a double-precision floating-point number.

The method declaration is followed by the method body, which in this example contains code that returns a placeholder value, so all that remains is to modify the code inside the braces delimiting the method body.

[Example 1-12](#) shows code that calculates the average instead of just returning 1.0.

Example 1-12. Calculating the average

```
return args.Select(numText => double.Parse(numText)).Average();
```

This relies on library functions for working with collections that are part of the set of features collectively known as LINQ, which is the subject of [Chapter 10](#). But just to describe quickly what's going on here, the `Select` method lets us apply an operation to every single item in a collection, and in this case, the operation I'm applying is the `double.Parse` method, a .NET runtime library function that converts a textual string containing a number into the native double-precision floating-point type. And then we push these transformed results through LINQ's `Average` method, which does the calculation for us.

With this in place, if I run my tests again, they will all pass. So apparently the code is working. However, I see a problem if I try to verify that informally by running the program, which I can do with this command:

```
./Averages/bin/Debug/net8.0/Averages 1 2 3 4 5
```

This just writes out `Hello, World!` to the screen. I've written and tested the code that performs the required calculation, but I've not yet connected that up to the program's entry point. The code that runs when the program starts lives in *Program.cs*, although there's nothing special about that filename. The program entry point can live in any file. In older versions of C#, you denoted the entry point by defining a `static` method called `Main`, as [Example 1-2](#) showed, and you can still do that with C# 12.0, but we normally use the newer, more succinct approach: we write a file that contains executable statements without putting them explicitly inside a method in a type, and the C# compiler will treat that as the entry point. (You're only allowed to have one file in your project written that way, because your program can have only one entry point.) If I replace the entire contents of *Program.cs* with the code shown in [Example 1-13](#), it will have the desired effect.

Example 1-13. Program entry point with arguments

```
using Averages;

Console.WriteLine(AverageCalculator.ArithmeticMean(args));
```


Notice that I've had to add a `using` directive—when you use this stripped-down program entry point syntax, the code in that file is not in any namespace by default, so I need to state that I want to use the class I defined in the `Averages` namespace. After that, this code invokes the method I wrote earlier, passing `args` as an argument, and then calls `Console.WriteLine` to display the result. When you use this style of program entry point, `args` is a special name—it's effectively an implicitly defined local variable that provides access to the command-line arguments. This will be an array of strings, with one entry for each argument. If you want to run the program again with the same arguments as before, run the `dotnet build` command first to rebuild it.

TIP

Some C-family languages include the filename of the program itself as the first argument, on the grounds that it's part of what the user typed at the command prompt. C# does not follow this convention. If the program is launched without arguments, the array's length will be 0. You might have noticed that the code does not cope well with that. Feel free to add a new test scenario that defines the relevant behavior, and to modify the program to match.

Unit Tests

Now that the program is working, I want to return to the tests, because they illustrate a C# feature that the main program does not. If you go back to [Example 1-3](#), it starts in a pretty ordinary way: we have a `using` directive and then a namespace declaration, for `Averages.Tests` this time, matching the test project name. But the class looks different. [Example 1-14](#) shows the relevant part of [Example 1-3](#).

Example 1-14. Test class with attribute

```
[TestClass]
public class WhenCalculatingAverages
{
```

Immediately before the class declaration is the text `[TestClass]`. This is an *attribute*. Attributes are annotations you can apply to classes, methods, and other features of the code. Most of them do nothing on their own—the compiler records the fact that the attribute is present in the compiled output, but that is all. Attributes are useful only when something goes looking for them, so they tend to be used by frameworks. In this case, I’m using Microsoft’s unit testing framework, and it goes looking for classes annotated with this `TestClass` attribute. It will ignore classes that do not have this annotation. Attributes are typically specific to a particular framework, and you can define your own, as we’ll see in [Chapter 14](#).

The two methods in the class are also annotated with attributes.

[Example 1-15](#) shows the relevant excerpts from [Example 1-3](#). The test runner will execute any methods marked with the `[TestMethod]` attribute.

Example 1-15. Annotated methods

```
[TestMethod]
public void SingleInputShouldProduceSameValueAsResult()
...

[TestMethod]
public void MultipleInputsShouldProduceAverageAsResult()
...
```

And with that, we’ve examined every element of a program and the test project that verifies that it works as intended.

Summary

You’ve now seen the basic structure of C# programs. I created a solution containing two projects, one for tests and one for the program itself. This was a simple example, so each project had only one or two source files of interest. Where necessary, these files began with `using` directives indicating which types the file uses. The program’s entry point used a stripped-down style, but the other two used a more conventional struc-

ture, containing a namespace declaration stating the namespace that the file populates, and a class containing one or more methods or other members, such as fields.

We will look at types and their members in much more detail in [Chapter 3](#), but first, [Chapter 2](#) will deal with the code that lives inside methods, where we express what we want our programs to do.

- [1](#) Native AOT therefore can't offer tiered compilation.
- [2](#) Older versions of the runtime we now call .NET were called .NET Core, and when this was rebranded as plain .NET, it skipped from 3.1 to 5.0 to emphasize the move away from .NET Framework, the latest version of which is 4.8.1.